



Structure & Capability Document

Tool	redeye 0.3.0
Category	Agentic SAST harness for autonomous vulnerability discovery
Output	SARIF 2.1.0 + Markdown + JSON manifest + PDF
Backends	Anthropic (CLI/SDK), OpenAI-compatible, AWS Bedrock, Google Vertex, Ollama
Generated	2026-06-30 10:27 UTC

Authorized use only. Findings are LLM-generated triage candidates that require human review.

1. Overview

RedEye pairs the deep multi-stage pipeline of an offline research harness with the operational layer of a CI/CD scanner -- the same tool runs as a researcher's deep dive and as a PR gate. It is multi-cloud by design: no single LLM provider is a dependency.

Three design choices drive finding quality:

- **Threat modeling before analysis** -- focuses the surface so lenses don't waste budget.
- **Multi-agent voting + single-pass validator** -- kills correlated false positives and obvious garbage.
- **Feedback loop** -- reviewer TP/FP marks persist locally and calibrate the next scan.

2. Pipeline architecture (9 core stages + sub-stages)

Each stage is a callable consuming a StageContext and returning a StageResult. The orchestrator owns flow; stages own content. Deterministic stages cost zero LLM tokens.

#	Stage	Skill	Type	Responsibility
S1	s1_attack_surface	attack_surface_mapper	LLM	Walks the repo, identifies entrypoints, sensitive sinks, auth boundaries; emits the attack-surface map. Owns repo-intake knobs.
S1b	s1b_structural	structural_index	Deterministic	Regex + AST ground-truth inventory: routes, sources, sinks, secrets. Folds external scanner reports into the map. Zero LLM cost.
S2	s2_threat_model	threat_modeler	LLM	STRIDE/OWASP threat model over the surface + structural evidence. Configurable threat/evidence caps and a threat baseline.
S3	s3_strategize	research_strategist	LLM	Chooses which research lenses to spend budget on, guided by the threat model.
S4	s4_research	research_lenses	LLM	The core finding generator: language, crypto, logic, access-control and IaC lenses produce candidate findings with taint flow.
S4b	s4b_grounding	(grounding pass)	Deterministic	Verifies every cited file path, line number and snippet against real code. Drops or downgrades hallucinated findings.
S5	s5_policy_gate	(policy gate)	Deterministic	Applies policy: required taint slots, severity floors, allow/deny rules.
S6	s6_adversarial	adversarial_reviewer	LLM	Adversarial refinement, then N-of-M multi-agent voting to kill correlated false positives.
S6b	s6b_validator	validator	LLM	Single-pass precision-filter validator: cheap auto-reject of obvious garbage.
S7	s7_dedupe	(dedupe)	Deterministic	Fingerprint-based de-duplication and local baseline filtering of already-accepted findings.
S8	s8_chain	exploit_strategist	LLM	Links related findings into multi-step attack chains.
S8b	s8b_poc	poc_gate	LLM	Demands a concrete proof-of-concept; demotes (or drops) findings with only hand-wavy PoCs.
S8c	s8c_verify	(verification)	Deterministic	Outcome verification: collapses grounding/taint/PoC/reachability/votes into a single auditable verdict.

#	Stage	Snail	Type	Responsibility
S9	s9_emit	(emitter)	Deterministic	Writes Markdown report, SARIF 2.1.0, and feeds the run manifest.

3. Hallucination-reduction layer

Seven deterministic and adversarial checks sit in front of every finding, so what reaches a reviewer cites real code and survived every cheap check the harness can run:

- **Structural pre-index (S1b)** -- real routes/sources/sinks so lenses reason from ground truth, not imagination.
- **Taint schema** -- every finding must declare source, sink and the path between them, or it isn't a finding.
- **Grounding pass (S4b)** -- cited paths/lines/snippets are verified against the actual file system.
- **Policy gate (S5)** -- required slots and severity floors enforced before spend on later stages.
- **Multi-agent voting (S6)** -- N-of-M voting kills correlated false positives across model families.
- **Validator auto-reject (S6b)** -- a cheap single-pass precision filter drops obvious garbage.
- **PoC gate (S8b) + outcome verification (S8c)** -- concrete exploit demanded; signals collapsed into a verdict.

4. Repo intake & file inventory (S1)

The `s1_attack_surface` stage owns intake. Knobs live under its params in `config.yaml` and have matching CLI flags; scalar CLI flags win, list flags merge (union) with config.

Knob	Behaviour
<code>exclude_dirs</code>	Directory names to skip during intake (merged with defaults).
<code>exclude_exts</code>	File extensions to drop; leading dot optional (e.g. <code>.min.js</code>).
<code>exclude_globs</code>	<code>fnmatch</code> globs against the repo-relative path.
<code>max_file_kb</code>	Skip files larger than N KB (combined with <code>max_file_bytes</code> ; smaller wins).
<code>follow_symlinks</code>	If false (default), symlinked files are skipped.
<code>dedupe_configs</code>	Drop byte-identical config files (<code>.yaml/.json/.toml/.ini/.env/...</code>).

5. Threat-model controls (S2)

The `s2_threat_model` stage is fully tunable: toggle it, cap output threats, subtract a threat baseline, and bound how much structural evidence is injected into the prompt.

Knob	Behaviour
<code>enabled</code>	false -> skip threat modeling entirely (no LLM call).
<code>max_threats</code>	Cap the number of emitted STRIDE entries.
<code>baseline</code>	Path to accepted-threat signatures (category asset) to subtract.
<code>max_document_chars</code>	Cap the attack-surface document fed into the prompt.
<code>max_modules / max_entry_points</code>	Evidence caps: modules and entry points injected.
<code>max_config_reps / max_api_artifacts</code>	Evidence caps: config reps and API artifacts injected.

6. External scanner ingestion (mapping enrichment)

RedEye consumes third-party scanner output -- **SARIF 2.1.0**, **Semgrep JSON**, and **generic JSON** -- and folds each finding's location into the structural map as a candidate sink hit. Provide reports via the `s1b_structural.params.external_scanners` list in `config.yaml` or the repeatable `--external-scan` CLI flag.

Mapping enrichment, not blind trust: an imported location becomes a candidate only -- it still has to clear grounding (S4b), voting (S6) and outcome verification (S8c) before it can reach the report. An ingestion summary is written to the Markdown report, the PDF report, and the run manifest.

7. Multi-cloud LLM backends

Backend	Module	Notes
Anthropic CLI	cli_claude	Local Claude CLI; no API key plumbing, great for laptops.
Anthropic SDK	sdk_anthropic	Direct Anthropic API (Claude models).
OpenAI-compatible	openai_compat	OpenAI and any OpenAI-compatible endpoint.
AWS Bedrock	bedrock	Managed frontier models in AWS.
Google Vertex	vertex	Gemini models via Vertex AI.
Ollama (local)	ollama	Fully local/offline models.
Mock	mock	Deterministic, zero-cost backend for demos and tests.

Profiles bundled: default, cli, full, mock, ollama_local. Each profile maps logical roles (surveyor, researcher, adversary, ...) to concrete models, temperatures, token and per-stage USD budgets.

8. CLI surface

Command	Purpose
redeye scan	Run the full 9-stage pipeline against one or many repos.
redeye estimate	Print scope and approximate USD cost. No LLM calls.
redeye doctor	Verify credentials and backend reachability for a profile.
redeye setup / init	Interactive setup; detect creds, pick a profile, write .env / config.yaml.
redeye baseline	accept / list / remove findings so they don't reappear.
redeye collect-feedback	Ingest TP/FP marks from a PR comment into the feedback store.

Key scan flags

Flag(s)	Effect
--repo / --repo-file	Single repo or CSV batch.
--profile / --preset	Cost model + roles; preset = pr ci deep quick one-flag combos.
--diff-only / --pr-base	Scan only files changed vs a base ref (PR scans).
--exclude-path / --exclude-dir / --exclude-ext / --exclude-glob	Intake exclusions (merged with config.yaml).
--max-files / --max-file-bytes / --max-file-kb / --max-total-bytes	DoS / cost limits on intake.
--follow-symlinks / --dedupe-configs	Symlink traversal; config-file de-duplication.
--external-scan PATH	Fold a SARIF/Semgrep/generic-JSON scanner report into the map (repeatable).
--strict-grounding / --require-poc	Drop (vs downgrade) ungrounded / PoC-less findings.
--store-findings / --use-feedback	Persist to and reuse the local feedback DB.
--pr-comment / --webhook-url	Emit a PR-comment Markdown; post a summary to Slack/Teams/Discord.

9. Outputs & artifacts

Artifact	Contents
Markdown report	Human-readable findings, quality metrics, structural inventory, external-scanner summary.
SARIF 2.1.0	Canonical machine output for code-scanning integrations.
run_manifest.json	Full audit record: every stage, artifacts, cost, hallucination metrics.
PR comment	GitHub-shaped Markdown for CI gating with TP/FP checkboxes.
PDF (this toolchain)	Styled per-scan report (build_report_pdf.py) and this capability doc.

10. Operational features

- **Presets** -- pr / ci / deep / quick collapse common flag combos; explicit flags always win.
- **PR scans** -- --diff-only against a base ref for fast, scoped CI gating.
- **Baseline** -- accept findings so they're filtered from future scans (.redeye-baseline.yaml).
- **Feedback loop** -- TP/FP marks persist to ~/.redeye/scans.db and re-enter S4 lens prompts.
- **Notifications** -- Slack/Teams/Discord/generic webhooks; GitHub PR-comment Markdown.
- **Cost governance** -- per-stage max_budget_usd ceilings; estimate command (intake-aware) for dry costing.
- **DoS protection** -- file/byte/KB caps and exclusions bound worst-case intake.
- **Secret redaction** -- the Markdown report masks known credential shapes and sensitive key=value pairs before hitting disk.

11. Configuration precedence

1) Explicit CLI flags > 2) ./config.yaml > 3) selected profile > 4) bundled defaults. Environment variables already set win over .env files. List-type intake exclusions are merged (config baseline + CLI additions).

12. Limitations & responsible use

- Findings are LLM-generated triage candidates, not confirmed vulnerabilities.
- Run only against code you own or are explicitly authorized to test.
- Severity, CWE and attack chains are starting points for human review.
- Deterministic checks reduce -- but do not eliminate -- false positives and negatives.

Generated from the redeye 0.3.0 codebase. Companion docs: SETUP_GUIDE, USER_GUIDE, architecture.md, SKILLS.md, configuration.md.